Platzhalter

Lo
go

Term paper

Sample title for a report

Author:	First name Last name
Matriculation No.:	xxxxxxx
Course of Studies:	Business Intelligence and Data Analytics
First examiner:	Prof. Dr. Elmar Wings
Submission date:	April 28, 2026

University of Applied Sciences Emden/Leer · Faculty of Technology · Mechanical Engineering Department
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Contents

Contents 3

List of Figures 7

List of Listings 9

1. Introduction 13

I. Domain Knowledge - Application 15

2. Application 17

II. Domain Knowledge - Hardware 19

3. Hardware XY 21
3.1. Further Readings . . . 21

4. HC-SR04 Ultraschallsensor 23
4.1. Allgemein . . . 23
4.2. HC-SR04 . . . 23
4.3. Spezifikation . . . 23
4.4. Bibliothek . . . 24
4.4.1. Beschreibung . . . 24
4.4.2. Installation . . . 24
4.4.3. Funktionen . . . 24
4.5. Beispiel . . . 24
4.5.1. Aufbau . . . 25
4.5.2. Beschreibung . . . 25
4.5.3. Code . . . 25
4.5.4. Dateien . . . 25
4.6. Kalibrierung . . . 25
4.7. Einfacher Code (ohne Bibliothek) . . . 26
4.8. Einfache Anwendung . . . 26
4.9. Tests . . . 27
4.9.1. Einfacher Funktionstest . . . 27
4.9.2. Alle Funktionen testen . . . 27
4.10. Weiterführende Literatur . . . 27

Bibliography 29

5. Ultraschallsensor HC-SR04 31
5.1. Allgemeines . . . 31
5.2. Spezifischer Sensor/Aktor . . . 31
5.3. Spezifikation . . . 32
5.4. Bibliothek . . . 32
5.4.1. Beschreibung . . . 32

5.4.2. Installation . . . 32
5.4.3. Funktionen . . . 32
5.5. Beispiel . . . 33
5.5.1. Handbuch . . . 33
5.5.2. Funktion vom Beispiel . . . 33
5.5.3. Code . . . 33
5.6. Kalibrierung . . . 34
5.7. Einfacher Code . . . 34
5.8. Einfache Anwendung . . . 35
5.9. Tests . . . 35
5.9.1. Einfacher Funktionstest . . . 35
5.9.2. Alle Funktionen testen . . . 35
5.10. Weiterführende Literatur . . . 35

Bibliography **37**

6. SG90 Servomotor **39**
6.1. Allgemeines . . . 39
6.2. Spezifischer Sensor/Aktor . . . 39
6.3. Spezifikation . . . 39
6.4. Bibliothek . . . 40
6.4.1. Beschreibung . . . 40
6.4.2. Installation . . . 40
6.4.3. Funktionen . . . 40
6.5. Beispiel . . . 40
6.5.1. Handbuch . . . 41
6.5.2. An einem Beispiel . . . 41
6.5.3. Code . . . 41
6.5.4. Dateien . . . 41
6.6. Kalibrierung . . . 41
6.7. Einfacher Code (ohne Bibliothek) . . . 42
6.8. Einfache Anwendung . . . 42
6.9. Tests . . . 42
6.9.1. Einfacher Funktionstest . . . 42
6.9.2. Alle Funktionen testen . . . 42
6.10. Weiterführende Literatur . . . 43

Bibliography **45**

III. Domain Knowledge - Tools **47**

7. Python Tool XY **49**

8. Hardware Tool XY **51**

IV. Developmenmt **53**

9. Flow Chart Development XY **55**

V. Monitoring **57**

10. Monitoring XY **59**

VI. Verification - Evaluation - Conclusion 61

11. Evaluation/Verification XY 63

12. To DoX Y 65

13. Conclusion XY 67

VII. Appendix 69

14. Bill of Material 71

15. SBOM 73

16. Package Example 75
16.1. Introduction . . . 75
16.2. Description . . . 75
16.3. Installation . . . 75
16.4. Example - Manual . . . 75
16.5. Example . . . 75
16.6. Example - Code . . . 75
16.7. Example - Files . . . 75
16.8. Further Reading . . . 75

VIII. Hints - Examples for LaTeX - Read, Use and Remove 77

17. Hints 79
17.1. Allgemeine mathematische Beschreibung Bézier-Kurve . . . 80

18. Verrundung mit einem Kreisbogen 83
18.1. Gleichungen . . . 83
18.2. Bewertung . . . 85
18.3. Examples . . . 86

19. Maple files 89
19.1. Geometries with Maple . . . 89
19.2. Geometry elements used . . . 89
19.3. Building the data structure . . . 90
19.4. Structure of a module . . . 90
19.4.1. General structure of a module . . . 92
19.4.2. Saving a module . . . 93
19.4.3. Verwendung eines Moduls . . . 93
19.4.4. Creating a Maple module . . . 93
19.5. Functions of the modules . . . 94
19.5.1. Module `MPoint` . . . 94
19.5.2. Module `MLine` . . . 95
19.5.3. Module `MArc` . . . 95
19.5.4. module `MBezier` . . . 96
19.5.5. Module `MPolygon` . . . 96
19.5.6. module `MGeoList` . . . 96
19.5.7. Module `MHermiteProblem` . . . 97
19.5.8. Module `MHermiteProblemSym` . . . 97
19.5.9. Module `MConstant` . . . 98
19.5.10. Module `MGeneralMath` . . . 98

19.5.11 Module Biarc 99
19.5.12 Module MBiarc 99
19.6. Programme Flowchart 100
19.6.1. Overall Flow 100
19.6.2. Verrundung der Kurve 100

20. Representation of Python Programs **103**

21. Representation of Programs written for Arduino Boards **105**

22. First Chapter **107**

23. CAGD **109**

A. drawings with tikz **111**

B. Criteria for a good LaTeX project **115**

Literature **117**

Index **119**

List of Figures

17.1. Bernstein polynomial of degree 3 81
17.2. Bézier curve for example 17.1 82

18.1. Smoothing a corner with the help of an arc - triangle 83
18.2. Angular change with blending arc 85
18.3. Curvature progression for a blending arc 85
18.4. Maximum range for a blending arc of a circle - $L(\varepsilon = const, \alpha)$ 86
18.5. Deviation when specifying the distance L when rounding with a circular arc 86

19.1. Programme flow chart „Corner rounding“ 101
19.2. Programme flowchart „Selection of the rounding strategies“ 102

List of Listings

20.1. „Hello World“ in Python – Variant 1 104

21.1. „Hello World“ in Python – Variant 1 106

Acronyms

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

1. Introduction

Flettner rotors are now widely used [1,2,3]. ...

The construction of a Flettner rotor for a water taxi [4] ...

External influences pose great challenges to [5,6] ...

Through the use of 3D printed parts, ...

In the second chapter, ...

Part I.

Domain Knowledge - Application

2. Application

Part II.

Domain Knowledge - Hardware

3. Hardware XY

3.1. Further Readings

4. HC-SR04 Ultraschallsensor

Einleitung

Dieses Kapitel beschreibt den HC-SR04 Ultraschallsensor im Detail, einschließlich seiner Hardware-Spezifikationen, der verwendeten Software-Bibliotheken sowie praktischer Anwendungs- und Codebeispiele.

WS:[**berrybase**; **sparkfun**; **arduino_newping**]

4.1. Allgemein

Der HC-SR04 ist ein Ultraschallsensor zur Entfernungsmessung. Der Sensor sendet Schallwellen aus und misst die Dauer, bis das Echo von einem reflektierenden Objekt zurückkommt. Anhand dieser Dauer lässt sich die exakte Distanz zum Objekt präzise berechnen. Typische Einsatzgebiete in der Praxis umfassen die Robotik (Hinderniserkennung), Automatisierungstechnik, Sicherheitssysteme sowie Füllstandsmessungen und Abstandswarner (z.B. Einparkhilfen).

4.2. HC-SR04

Der HC-SR04-Sensor besteht grundlegend aus drei Hauptkomponenten: einem Ultraschallsender (Transmitter), einem Ultraschallempfänger (Receiver) und einer integrierten Steuerschaltung. Er verfügt über vier Pins zur Verbindung mit der Hardware: `VCC` (Stromversorgung), `TRIG` (Triggereingang), `ECHO` (Ausgang für das Messergebnis) und `GND` (Masse). Um den Sensor mit einem Entwicklungsboard anzusteuern, wird ein mindestens 10 Mikrosekunden langer HIGH-Impuls an den `TRIG`-Pin gesendet. Daraufhin sendet der Sensor vollautomatisch acht Ultraschallimpulse mit einer Frequenz von 40 kHz aus und setzt parallel den `ECHO`-Pin auf HIGH. Sobald das Ultraschallsignal reflektiert wird und zum Empfänger zurückkehrt, fällt der `ECHO`-Pin wieder auf LOW. Die Dauer, die der `ECHO`-Pin auf HIGH stand, entspricht der Schalllaufzeit.

4.3. Spezifikation

- **Datenblatt referenzieren:** Als primäre Referenzen für die physikalischen Grenzen dienen die offiziellen Datenblätter von Sparkfun, BerryBase und Handson Technology.
- **Extrahierte Informationen:**
  - Betriebsspannung: 5 V DC
  - Betriebsstrom: ca. 15 mA (Ruhestrom < 2 mA)
  - Arbeitsfrequenz: 40 kHz
  - Messbereich: 2 cm bis 400 cm (Herstellerabhängig spezifiziert bis max. 450 cm)
  - Messgenauigkeit: bis zu 3 mm
  - Effektiver Messwinkel: < 15 Grad
  - Trigger-Signal: 10 µs TTL-Impuls

- **Schaltplan:** In einem typischen Schaltungsaufbau wird VCC des Sensors mit dem 5V-Ausgang des Mikrocontrollers verbunden und GND mit der gemeinsamen Masse. TRIG wird an einen digitalen Ausgangspin und ECHO an einen digitalen Eingangspin des Mikrocontrollers angeschlossen.

4.4. Bibliothek

4.4.1. Beschreibung

Für die effiziente Ansteuerung des HC-SR04 wird in der Regel die `NewPing`-Bibliothek von Tim Eckel verwendet. Diese Bibliothek bietet erweiterte Features wie Timer-Interrupts für blockierungsfreie ereignisgesteuerte Sketche sowie integrierte digitale Filterung.

4.4.2. Installation

Die Installation kann direkt in der Arduino IDE über den internen Bibliotheksverwalter erfolgen. Alternativ kann sie als .ZIP-Datei von der Entwicklerseite bezogen und manuell in die Entwicklungsumgebung importiert werden.

- **Data types:** Die Bibliothek operiert primär mit `unsigned int` als Rückgabedatentyp, um Messzeiten in Mikrosekunden oder Längen in cm/inch speichereffizient zu verarbeiten.

- **Data structure:** Jeder verbaute Sensor wird als dediziertes C++ Objekt der Klasse `NewPing` instanziiert.

- **Data quantity:** Das Softwaredesign der Bibliothek ermöglicht das gleichzeitige, asynchrone Pingen von bis zu 15 Sensoren durch den Einsatz von Hardware-Timern. Pro Sensor können absolut zuverlässig bis zu 30 Pings pro Sekunde durchgeführt werden.

- **Data quality:** Die integrierte Funktion `ping_median(iterations)` erhöht die Datenqualität drastisch. Sie führt mehrere Messungen hintereinander aus und liefert den Medianwert zurück, wodurch zufällige Messausreißer effektiv herausgefiltert werden.

4.4.3. Funktionen

Die Bibliothek stellt primär folgende Methoden zur Verfügung:

- `sonar.ping()`: Sendet ein Ultraschallsignal aus und gibt die exakte Laufzeit in Mikrosekunden zurück.

- `sonar.ping_cm()`: Führt eine Messung durch und gibt die direkt berechnete Distanz in ganzen Zentimetern zurück.

- `sonar.ping_median(iterations)`: Führt eine definierte Anzahl an Messungen durch und verwirft ungültige Ausreißer.

4.5. Beispiel

In diesem Abschnitt wird ein einfaches Anwendungsbeispiel zur Distanzmessung mit der NewPing-Bibliothek detailliert beschrieben.

4.5.1. Aufbau

Verbinden Sie die Stromversorgungs-Pins (VCC und GND) des HC-SR04 mit den 5V- und GND-Pins des Arduinos. Verbinden Sie den TRIG-Pin des Sensors mit dem digitalen Pin 12 und den ECHO-Pin mit dem digitalen Pin 11 auf dem Board. Verbinden Sie den Arduino über USB mit dem Rechner, kompilieren Sie das Skript und öffnen Sie den seriellen Monitor.

4.5.2. Beschreibung

Der Code beginnt mit der Einbindung der Bibliothek und der Definition der Pins. Das `NewPing`-Objekt `sonar` wird mit Trigger-Pin, Echo-Pin und einer definierten maximalen Messdistanz initialisiert. In der Hauptschleife (`loop`) wird im Abstand von 50 Millisekunden ein Ping ausgelöst. Die zurückgegebene Laufzeit in Mikrosekunden wird durch den standardisierten Teilerfaktor 58 dividiert, um die korrekte Entfernung in Zentimetern zu errechnen.

4.5.3. Code

```
#include <NewPing.h>

#define TRIGGER_PIN  12  // Arduino Pin verbunden mit TRIG
#define ECHO_PIN     11  // Arduino Pin verbunden mit ECHO
#define MAX_DISTANCE 400 // Maximale Reichweite in cm

// Initialisierung des Objektes
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

void setup() {
 Serial.begin(9600);
}

void loop() {
 delay(50); // Empfohlene Mindestwartezeit zwischen Pings (ca. 20 pings/sec)
 unsigned int uS = sonar.ping(); // Pingen und Zeit in Mikrosekunden erhalten

 Serial.print("Distanz: ");
 Serial.print(uS / 58); // Formel: Mikrosekunden / 58 = Zentimeter
 Serial.println(" cm");
}
```

4.5.4. Dateien

Für das reibungslose Kompilieren dieses Beispiels wird zwingend die Header-Datei `NewPing.h` benötigt, die auf dem System installiert sein muss.

4.6. Kalibrierung

Die Messung der Ultraschall-Laufzeit basiert physikalisch auf der Ausbreitung von Schall in der Luft (Schallgeschwindigkeit), welche bei ca. 20 °C exakt 343 m/s beträgt. Die standardisierte Berechnungsformel lautet:

$$\text{Distanz (cm)} = \frac{\text{Laufzeit (µs)}}{58}$$

oder als Rohberechnung:

$$\text{Distanz} = \frac{\text{Laufzeit} \times \text{Schallgeschwindigkeit}}{2}$$

(Dividiert durch 2, da der Schall die Strecke zum Objekt und wieder zurück zurücklegen muss). Da die Schallgeschwindigkeit temperaturabhängig ist, kann für hochpräzise Kalibrierungen ein zusätzlicher Temperatursensor in die Schaltung integriert werden, um den Teilerfaktor in Echtzeit anzupassen.

4.7. Einfacher Code (ohne Bibliothek)

Um das nackte Hardware-Verhalten des Sensors ohne Software-Bibliothek zu testen, reicht folgendes natives C++ Konstrukt:

```
int trig = 11;
int echo = 12;

void setup() {
 pinMode(trig, OUTPUT);
 pinMode(echo, INPUT);
 Serial.begin(9600);
}

void loop() {
 // Trigger-Impuls erzeugen (10 Mikrosekunden)
 digitalWrite(trig, LOW);
 delayMicroseconds(2);
 digitalWrite(trig, HIGH);
 delayMicroseconds(10);
 digitalWrite(trig, LOW);

 // Zeit messen, bis Echo-Pin auf LOW fällt
 long duration = pulseIn(echo, HIGH);
 float distance = duration / 58.0;

 Serial.print("Distanz: ");
 Serial.print(distance);
 Serial.println(" cm");
 delay(100);
}
```

4.8. Einfache Anwendung

Eine gängige, einfache Anwendung für den HC-SR04 ist ein smarter Garagen-Einparkassistent. Das Sensormodul wird auf Stoßstangenhöhe an der Garagenwand angebracht. Nähert sich das Fahrzeug auf unter 50 cm, warnt eine LED-Matrix (Gelb). Bei weniger als 15 cm Entfernung leuchtet die Matrix Rot und ein aktiver Summer gibt ein akustisches Stopp-Signal. Die `NewPing`-Bibliothek gewährt hierbei die Sicherheit, dass es durch Fehlmessungen oder Reflektionen an Autokonturen nicht zu Softwarehängern kommt.

4.9. Tests

4.9.1. Einfacher Funktionstest

Zur Überprüfung der grundlegenden Schaltung sollte nach dem Upload ein harter, planer Gegenstand (z. B. ein Buchkasten) in exakt 20 cm Abstand frontal vor den Sensor gehalten werden. Der Code sollte im Serial Monitor den konstanten Wert von 20 cm anzeigen, mit einer minimalen Abweichung von höchstens ± 3 mm.

4.9.2. Alle Funktionen testen

Ein umfassender Belastungstest für Sensor und Code umfasst folgende Kriterien: 1. **Minimaldistanz:** Annäherung auf 2 cm – prüfen, ob der Messwert linear bleibt oder verfälscht wird. 2. **Maximaldistanz:** Ausrichtung auf eine flache, große Wand in über 4 Metern Entfernung (Fläche muss > 0.5 qm betragen), um das Reichweitenlimit auszutesten. 3. **Messwinkel:** Bewegungen von Objekten in Winkeln von $> 15°$ zur Sensorachse dürfen keine gültigen Echos mehr erzeugen. 4. **Filterung:** Überprüfung von `sonar.ping_median(5)`, indem der Sensor auf weiche, schallschluckende Stoffe oder extrem unebene Oberflächen gerichtet wird, bei denen herkömmliche Codes stark springende Werte ausgeben.

4.10. Weiterführende Literatur

Bibliography

[1] BerryBase (2024): *Datasheet for HC-SR04 Ultraschall Sensor.*

[2] SparkFun Electronics: *Ultrasonic Ranging Module HC - SR04.*

[3] Handson Technology: *HC-SR04 Ultrasonic Sensor Module User Guide.*

[4] Arduino Documentation: *NewPing Library Reference.*

[5] Tim Eckel / Scribd: *Arduino NewPing Library Documentation.*

[6] Teach Engineering: *NewPing Instructions and Code for Distance Sensing.*

5. Ultraschallsensor HC-SR04

Einleitung

Dieses Kapitel befasst sich mit der berührungslosen Abstandsmessung mithilfe von Sensoren. Zunächst wird ein allgemeiner Überblick über gängige Sensortechnologien zur Distanzmessung gegeben. Anschließend wird der in diesem Projekt verwendete Ultraschallsensor HC-SR04 detailliert beschrieben, einschließlich seiner technischen Spezifikationen und der softwareseitigen Implementierung unter Verwendung ausgewählter C++ Bibliotheken.

WS:[**SparkFun:2020**; **Eckel:2023**; **ArduinoReference:20**

5.1. Allgemeines

Sensoren zur Abstandsmessung sind wichtige Komponenten in der modernen Automatisierungstechnik, Robotik und Fahrzeugtechnik [**Fraden:2016**]. Sie ermöglichen es Systemen, eine Umgebung räumlich zu erfassen. Grundsätzlich lassen sich diese Sensoren in verschiedene Kategorien einteilen:

- **Infrarotsensoren:** Nutzen das Reflexionsverhalten von Licht oder messen die Lichtlaufzeit (Time-of-Flight, ToF). Sie sind extrem schnell und präzise, jedoch stark abhängig von den Oberflächeneigenschaften (Farbe, Reflexionsgrad) des Zielobjekts und anfällig für Fremdlichteinstrahlung [**Fraden:2016**].

- **Radarsensoren:** Arbeiten mit elektromagnetischen Wellen im Radio- oder Mikrowellenbereich. Sie sind unempfindlich gegenüber Schmutz, Nebel oder Lichtverhältnissen, jedoch oft komplex in der Auswertung und teurer in der Anschaffung [**Fraden:2016**].

- **Ultraschallsensoren:** Arbeiten nach dem Echo-Lot-Prinzip (Schalllaufzeitmessung). Sie senden Schallwellen aus und messen die Zeit, bis das Echo zurückkehrt [**SparkFun:2020**]. Sie sind unabhängig von der Farbe oder optischen Transparenz eines Objekts, haben jedoch ihre Grenze bei schallabsorbierenden Materialien (z. B. Schaumstoff) [**Fraden:2016**].

5.2. Spezifischer Sensor/Aktor

In diesem Projekt kommt der Ultraschallsensor **HC-SR04** zum Einsatz. Er wandelt die gemessene Entfernung in ein PWM-Signal um, welches am Ausgang zur Verfügung steht [**KT:2012**].
Das Auslösen eines Messzyklus geschieht durch eine fallende Flanke am `TRIG`-Eingang für mindestens 10 µs. Das Modul sendet darauf nach ca. 250 µs ein 40 kHz Burst-Signal für die Dauer von 200 µs aus. Danach geht der `ECHO`-Ausgang sofort auf HIGH-Pegel und das Modul wartet auf den Empfang des Echos. Wird dieses detektiert, fällt der Ausgang auf LOW-Pegel. Wird kein Echo detektiert, verweilt der Ausgang für insgesamt 200 ms auf HIGH-Pegel und zeigt so eine erfolglose Messung an. Ein vollständiges Messintervall hat eine Dauer von 20 ms, sodass maximal 50 Messungen pro Sekunde durchgeführt werden können [**KT:2012**].

5.3. Spezifikation

- **Datenblatt:** Die elektrischen und physikalischen Parameter beziehen sich auf die offiziellen Spezifikationen von KT-elektronic [**KT:2012**].
  - Betriebsspannung (`VCC`): +5 V DC
  - Stromaufnahme: < 2 mA
  - Messbare Distanz: 2 cm bis ca. 300 cm
  - Messauflösung: 3 mm
  - Messintervall: 20 ms (maximal 50 Messungen pro Sekunde)
  - Messwinkel: Bis zu 45° bei kurzen Distanzen (< 1 m), Sendekegel von 15° bei maximaler Distanz
  - Abmessungen (L x B x T): 45 x 21 x 18 mm
- **Schaltplan:** Das Modul besitzt vier Pins. Der `VCC`-Pin (Pin 1) wird mit der 5V-Spannungsversorgung des Mikrocontrollers verbunden, der `GND`-Pin (Pin 4) mit dem `GND`-Pin des Microcontrollers. Der `TRIG`-Eingang (Pin 2) und der `ECHO`-Ausgang (Pin 3) arbeiten mit TTL-Pegel. Hinweis bei 3.3V Mikrocontrollern: Da der `ECHO`-Pin ein 5V-Signal liefert, muss dieser zwingend durch einen Spannungsteiler an den 3.3V-Eingang des Mikrocontrollers angepasst werden [**KT:2012**].

5.4. Bibliothek

5.4.1. Beschreibung

Für die Ansteuerung des Sensors wird die externe C++ Bibliothek **NewPing** von Tim Eckel verwendet [**Eckel:2023**].

5.4.2. Installation

Die Bibliothek kann direkt über den internen Bibliotheksverwalter der Arduino IDE installiert werden [**Eckel:2023**].

- **Data types:** Messzeiten in Mikrosekunden und Distanzen in Zentimetern werden als `unsigned int` deklariert, um Speicherplatz zu sparen und negative Distanzen auszuschließen.
- **Data structure:** Der Sensor wird als Objekt der Klasse `NewPing` instanziiert.
- **Data quantity:** Das Softwaredesign ist ressourcenschonend konzipiert. Es können bis zu 15 Ultraschallsensoren gleichzeitig und asynchron über ein einziges Board betrieben werden, bei einer Abtastrate von bis zu 30 Pings pro Sekunde je Sensor.
- **Data quality:** Die Bibliothek bietet integrierte digitale Filter. Die Funktion `ping_median(iterations)` verarbeitet mehrere Messwerte zu einem Median, was Rauschen und fehlerhafte Echos effektiv aus dem Datensatz entfernt.

5.4.3. Funktionen

Die wichtigsten Befehle der `NewPing`-Bibliothek umfassen [**Eckel:2023**]:

- `NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE)`: Konstruktor zur Erstellung des Sensor-Objekts.

- `sonar.ping()`: Sendet einen Ping und gibt die gemessene Laufzeit in Mikrosekunden (µs) zurück.
- `sonar.ping_cm()`: Führt eine Messung durch und gibt die in Zentimeter (cm) umgerechnete Distanz zurück.
- `sonar.ping_median(int iterations)`: Führt die angegebene Anzahl an Pings durch, verwirft Ausreißer und gibt die mediane Laufzeit zurück.

5.5. Beispiel

In diesem Abschnitt wird ein einfaches, eigenständiges Beispiel im Detail beschrieben, welches die Funktionsweise und Integration der Bibliothek in C++ verdeutlicht.

5.5.1. Handbuch

1. Verbinden Sie `VCC` und `GND` des HC-SR04 mit 5V und GND des Boards. 2. Verbinden Sie den `TRIG`-Pin mit Pin 12 und den `ECHO`-Pin mit Pin 11. 3. Öffnen Sie die Arduino IDE, kompilieren Sie den Code und laden Sie ihn hoch. 4. Öffnen Sie den Seriellen Monitor (Baudrate 115200)[**SparkFun:2020**].

5.5.2. Funktion vom Beispiel

Das Skript bindet die `NewPing.h` ein. Über `#define` werden Pins und der maximale Messbereich (200 cm) definiert. In der Hauptschleife (`loop()`) stellt `delay(50)` sicher, dass zwischen zwei Pings gewartet wird, um Echosüberlappungen zu verhindern [**Eckel:2023**]. Anschließend wird `ping_median(5)` aufgerufen. Der Rückgabewert wird durch `US_ROUNDTRIP_CM` geteilt, um Zentimeter zu erhalten [**Eckel:2023**].

5.5.3. Code

```
#include <NewPing.h>

/** @brief Digitaler Pin für das Trigger-Signal */
#define TRIGGER_PIN  12

/** @brief Digitaler Pin für das Echo-Signal */
#define ECHO_PIN    11

/** @brief Maximale Distanz, auf die der Sensor reagieren soll (in cm) */
#define MAX_DISTANCE 200

/** * @brief Initialisierung des NewPing-Objekts.
* @param TRIGGER_PIN Der konfigurierte Sende-Pin.
* @param ECHO_PIN Der konfigurierte Empfangs-Pin.
* @param MAX_DISTANCE Die definierte maximale Reichweite.
*/
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);

void setup() {
 Serial.begin(115200);
 Serial.println("System gestartet. Beginne Messung...");
}
```

```
void loop() {
 delay(50); // Pause zwischen Pings (min. 29ms empfohlen laut Doku)

 /** * @brief Holt den Median aus 5 Messungen (in Mikrosekunden). */
 unsigned int uS = sonar.ping_median(5);

 Serial.print("Gefilterte Distanz: ");

 if (uS == 0) {
  Serial.println("Außerhalb der Reichweite");
 } else {
  // Umrechnung von Mikrosekunden in Zentimeter
  Serial.print(uS / US_ROUNDTRIP_CM);
  Serial.println(" cm");
 }
}
```

5.6. Kalibrierung

Die Ultraschallmessung beruht auf der Ausbreitungsgeschwindigkeit von Schall in der Luft. Die Standardberechnung der Bibliothek geht von einer konstanten Schallgeschwindigkeit von $c \approx 343\,\mathrm{m/s}$ bei $20\,^\circ\mathrm{C}$ aus [**Fraden:2016**] . Da die Schallgeschwindigkeit temperaturabhängig ist, ändert sie sich um ca. $0,6\,\mathrm{m/s}$ pro Grad Celsius [**Fraden:2016**]. Für hochpräzise Anwendungen muss das System kalibriert werden (z.B. über einen DHT22-Sensor). Die dynamische Berechnung lautet:

$$\text{Distanz} = \frac{\text{Laufzeit}}{2} \times (331,3 + 0,606 \times \text{Temperatur in } ^\circ\text{C})$$

5.7. Einfacher Code

Der folgende Code demonstriert eine Abstandsmessung ohne Bibliotheken und nutzt `pulseIn()`.

```
/**
* @file   HCSR04_Native_Example.ino
* @brief  Native Distanzmessung ohne externe Bibliotheken.
*/

const int TRIG_PIN = 12;
const int ECHO_PIN = 11;

void setup() {
 pinMode(TRIG_PIN, OUTPUT);
 pinMode(ECHO_PIN, INPUT);
 Serial.begin(115200);
}

void loop() {
 digitalWrite(TRIG_PIN, LOW);
 delayMicroseconds(2);

 // 10 Mikrosekunden HIGH-Impuls (laut Datenblatt)
 digitalWrite(TRIG_PIN, HIGH);
```

```
 delayMicroseconds(10);
 digitalWrite(TRIG_PIN, LOW);

 /** * @brief pulseIn misst die Länge des ECHO-Impulses. */
 long duration = pulseIn(ECHO_PIN, HIGH);

 // Berechnung der Distanz (Laufzeit in us / 58)
 float distance = duration / 58.0;

 Serial.print("Gemessene Distanz: ");
 Serial.print(distance);
 Serial.println(" cm");

 delay(100);
}
```

5.8. Einfache Anwendung

Eine anschauliche Anwendung für den HC-SR04 ist eine automatisierte Einparkhilfe. Fährt ein Fahrzeug heran, liest der Sensor die Distanz. Unterschreitet der `distance`-Wert 50 cm, schaltet das System eine Warn-LED ein. Fällt der Wert auf unter 10 cm, wird ein Buzzer getriggert [**Eckel:2023**].

5.9. Tests

5.9.1. Einfacher Funktionstest

Ein Gegenstand wird in exakt 15 cm Entfernung parallel zur Sensorfront platziert. Der ausgegebene Wert muss stabil bei 15 cm liegen. Die Schwankung sollte mit `sonar.ping_median()` nicht mehr als $\pm 0,5$ cm betragen [**SparkFun:2020**].

5.9.2. Alle Funktionen testen

Ein tiefgreifender Test umfasst:

- **Minimaldistanz:** Annäherung auf < 2 cm, um das Verhalten in der Blindzone zu testen [**SparkFun:2020**].
- **Maximaldistanz:** Ausrichten auf eine Wand in > 400 cm Entfernung, um den definierten Timeout zu verifizieren [**Eckel:2023**].
- **Winkeltest:** Bewegung von Objekten in Winkeln $> 15°$ zur Überprüfung des Abstrahlkegels [**SparkFun:2020**].

5.10. Weiterführende Literatur

Bibliography

[1] SparkFun Electronics / ElecFreaks: *HC-SR04 Ultrasonic Sensor Datasheet*. [Online; abgerufen für Spezifikationen und Timing-Diagramme].

[2] Eckel, Tim (2023): *Arduino NewPing Library Documentation*. GitHub/Bitbucket. [Online; abgerufen für API-Befehle und Event-Driven Architecture].

[3] Arduino LLC (2024): *Arduino Language Reference - pulseIn()*. [Online; abgerufen für native Mikrocontroller-Befehle].

[4] Fraden, Jacob (2016): *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer. [Abgerufen für akustische Grundlagen und Schallgeschwindigkeit].

6. SG90 Servomotor

Einleitung

Dieses Kapitel beschreibt den mechanischen Aktor des Systems, den Micro-Servomotor SG90. Es werden die technischen Spezifikationen, die hardwareseitige Einbindung sowie die softwareseitige Ansteuerung über die Standard-Bibliotheken detailliert erläutert.

WS:[**towerpro**; **arduino_servo**; **components101**]

6.1. Allgemeines

Ein Servomotor ist ein geschlossenes mechatronisches System (Closed-Loop), das für präzise Winkelpositionierungen eingesetzt wird. Im Gegensatz zu kontinuierlich drehenden Gleichstrommotoren besteht ein Standard-Servo aus einem DC-Motor, einem Untersetzungsgetriebe, einem Potentiometer zur Positionserfassung und einer Steuerelektronik. Der Mikrocontroller sendet ein Pulsweitenmodulations-Signal (PWM), welches die Steuerelektronik auswertet und den Motor exakt auf den gewünschten Winkel dreht und dort aktiv hält. Servomotoren findet man vor allem im RC-Modellbau (z.B. Lenkung, Ruder, Klappen), in Robotergelenken sowie in Schwenkmechanismen für Kameras oder Sensoren, bei denen wiederholgenaue und stabile Positionen erforderlich sind

6.2. Spezifischer Sensor/Aktor

Der TowerPro SG90 ist einer der weltweit am häufigsten verwendeten Micro-Servomotoren für Maker-Projekte. Er zeichnet sich durch sein extrem geringes Gewicht, seine kompakte Bauform und sein Nylon-Zahnradgetriebe aus. Der Aktor verfügt über ein dreipoliges Anschlusskabel: Braun für die Masse (`GND`), Rot für die Versorgungsspannung (`VCC`) und Orange für das PWM-Steuersignal (`Signal`). Der SG90 hat einen mechanischen Rotationsspielraum von 180 Grad (0° bis 180°). Gesteuert wird er über ein 50-Hz-PWM-Signal (20 ms Periodendauer). Eine Pulsweite von 1 ms entspricht theoretisch 0°, 1,5 ms entsprechen 90° (Mittelstellung) und 2 ms entsprechen 180°.

6.3. Spezifikation

- **Datenblatt referenzieren:** Die physikalischen und elektrischen Kennwerte beziehen sich auf die offiziellen Spezifikationen von TowerPro und das Datenblatt von Components101.
- **Extrahierte Informationen:**
  - Betriebsspannung: 4,8 V bis 6,0 V DC (Optimaler Betrieb bei 5 V)
  - Stellmoment (Stall Torque): 1,8 kg/cm (bei 4,8 V)
  - Stellgeschwindigkeit (ohne Last): 0,1 Sekunden / 60 Grad (bei 4,8 V)
  - Gewicht: 9 Gramm
  - Getriebetyp: Kunststoff (Nylon)
  - Steuersignal: PWM (50 Hz / 20 ms Periode)

- **Schaltplan:** Das rote Kabel (`VCC`) des SG90 wird mit dem 5V-Ausgang (VUSB) des Systems verbunden, um ausreichend Leistung für den Motor bereitzustellen. Das braune Kabel (`GND`) wird mit der gemeinsamen Masse verbunden. Das orange Kabel (`Signal`) wird an einen digitalen PWM-fähigen Pin (z.B. Pin 9) des Mikrocontrollers angeschlossen. *Hinweis:* Obwohl der Arduino Nano 33 BLE mit 3,3V-Logik arbeitet, reicht die 3,3V-Signalspannung am PWM-Pin völlig aus, um den 5V-Servomotor zuverlässig zu triggern.

6.4. Bibliothek

6.4.1. Beschreibung

Für die Ansteuerung des SG90 wird in der Arduino-Umgebung die integrierte `Servo`-Bibliothek genutzt. Dadurch entfällt für den Programmierer die aufwändige Erzeugung des exakten 50-Hz-PWM-Signals, und der Motor kann bequem über die direkte Vorgabe eines Winkelwertes von 0 bis 180 Grad angesteuert werden.

6.4.2. Installation

Die `Servo`-Bibliothek gehört zu den Kern-Bibliotheken der Arduino IDE und muss nicht separat heruntergeladen werden. Sie ist direkt nach der Installation der Entwicklungsumgebung unter `#include <Servo.h>` verfügbar.

- **Data types:** Die Zielwinkel werden als `int` (Integer/Ganzzahlen) übergeben. Für präzisere Steuerungen können Pulsweiten in Mikrosekunden ebenfalls als `int` übergeben werden.
- **Data structure:** Jeder Servomotor wird im Code als Objekt der Klasse `Servo` deklariert (z.B. `Servo meinServo;`).
- **Data quantity:** Auf Standard-Arduino-Boards unterstützt die Bibliothek bis zu 12 Servos gleichzeitig; auf speziellen Architekturen (wie der Mbed-OS Architektur des Nano 33 BLE) kann dies variieren, reicht jedoch für mehrere Aktoren vollkommen aus.
- **Data quality:** Die Bibliothek nutzt Hardware-Timer des Mikrocontrollers (statt softwareseitiger Delays), wodurch das PWM-Signal im Hintergrund extrem stabil und jitter-frei generiert wird, ohne den restlichen Programmablauf zu blockieren.

6.4.3. Funktionen

Die wichtigsten Methoden der Bibliothek sind:

- `attach(pin)`: Verbindet das Servo-Objekt mit einem bestimmten digitalen Pin.
- `write(angle)`: Dreht den Servo auf den entsprechenden Winkel (0° bis 180°).
- `writeMicroseconds(uS)`: Feinkontrolle des Servos durch direkte Angabe der Pulsweite in Mikrosekunden.
- `read()`: Gibt den aktuellen, zuletzt geschriebenen Winkel des Servos zurück.

6.5. Beispiel

Dieser Abschnitt zeigt ein elementares Beispiel ("Sweep"), bei dem der Servo kontinuierlich von 0 auf 180 Grad und wieder zurück schwenkt.

6.5.1. Handbuch

Schließen Sie das rote Kabel an 5V, das braune an GND und das orange Signalkabel an Pin 9 des Arduinos an. Kompilieren Sie das Programm und laden Sie es auf das Board. Der Arm des Servomotors sollte nun anfangen, langsam und gleichmäßig hin und her zu wischen.

6.5.2. An einem Beispiel

Das Skript bindet zunächst die Bibliothek ein und erstellt ein Servo-Objekt. In der `setup()`-Routine wird der Motor an Pin 9 gebunden. In der `loop()`-Schleife sorgen zwei `for`-Schleifen für die Bewegung. Die erste Schleife erhöht den Winkel schrittweise von 0 auf 180, gefolgt von einer kurzen Pause (`delay`) von 15 ms pro Grad, um dem Motor Zeit zu geben, die physikalische Position zu erreichen. Die zweite Schleife führt die Rückwärtsbewegung aus.

6.5.3. Code

```
#include <Servo.h>

Servo radarServo;  // Erstellt ein Servo-Objekt
int pos = 0;     // Variable zum Speichern der aktuellen Position

void setup() {
 radarServo.attach(9);  // Verbindet den Servo an Pin 9 mit dem Objekt
}

void loop() {
 // Schwenkt von 0 Grad auf 180 Grad
 for (pos = 0; pos <= 180; pos += 1) {
  radarServo.write(pos);
  delay(15);  // Wartet 15ms für das Erreichen der Position
 }
 // Schwenkt von 180 Grad zurück auf 0 Grad
 for (pos = 180; pos >= 0; pos -= 1) {
  radarServo.write(pos);
  delay(15);
 }
}
```

6.5.4. Dateien

Für das Ausführen des Codes wird ausschließlich die Standard-Headerdatei `Servo.h` benötigt, welche in der Arduino IDE vorinstalliert ist.

6.6. Kalibrierung

Oftmals weichen günstige SG90-Motoren mechanisch von den theoretischen PWM-Werten ab (1000 µs bis 2000 µs). Wird der Motor mit `write(0)` oder `write(180)` an seine physischen Grenzen gezwungen, kann er brummen, vibrieren und überhitzen (Stalling). Zur Kalibrierung bietet die `attach()`-Funktion optionale Parameter für die minimalen und maximalen Mikrosekunden: `radarServo.attach(9, 500, 2400);`. Diese Werte müssen durch iteratives Ausprobieren (mit `writeMicroseconds()`) an den individuellen Motor angepasst werden, bis der volle 180°-Winkel ohne mechanisches Blockieren erreicht ist.

6.7. Einfacher Code (ohne Bibliothek)

Um zu verdeutlichen, wie die Hardware funktioniert, zeigt dieser Code die Ansteuerung des SG90 durch reine Bit-Manipulation (Blink-Prinzip), ohne die Servo-Bibliothek zu verwenden:

```
int servoPin = 9;

void setup() {
 pinMode(servoPin, OUTPUT);
}

void loop() {
 // Bewegung auf ca. 90 Grad (1.5ms HIGH-Puls)
 digitalWrite(servoPin, HIGH);
 delayMicroseconds(1500); // 1.5 Millisekunden
 digitalWrite(servoPin, LOW);

 // Auffüllen der restlichen 20ms Periode (50 Hz)
 delayMicroseconds(18500);
}
```

6.8. Einfache Anwendung

Eine klassische Anwendung für den SG90 ist die Basis eines **Ultraschall-Radarsystems** (wie im vorliegenden Projekt). Der Ultraschallsensor HC-SR04 wird auf der Achse des Servos montiert. Der Servo tastet systematisch in 1-Grad-Schritten die Umgebung vor sich ab (0° bis 180°). Bei jedem Schritt wird eine Entfernungsmessung durchgeführt. Diese Kombination aus Aktor-Winkel und Sensor-Distanz ermöglicht es der Software (z. B. in Processing), eine zweidimensionale "Radarkarte" der Umgebung auf dem Bildschirm zu zeichnen.

6.9. Tests

6.9.1. Einfacher Funktionstest

Mit dem "Sweep"-Code aus Abschnitt 6.3 lässt sich leicht prüfen, ob der Motor korrekt verkabelt ist. Der Motor muss ruckelfrei und flüssig schwenken. Ein stotternder Motor oder ein Neustart des Arduinos während der Bewegung deutet auf eine unzureichende Stromversorgung (VCC-Abfall) hin.

6.9.2. Alle Funktionen testen

Ein vollständiger Systemtest umfasst: 1. **Grenztest:** Anfahren von 0° und 180°. Der Motor darf in diesen Endlagen nicht surren oder ungewöhnlich warm werden. 2. **Geschwindigkeitstest:** Befehle `write(0)` und sofort danach `write(180)` ohne Delay. Messung, ob der Motor die physikalische Herstellerangabe von 0,3 Sekunden für diesen 180°-Weg einhalten kann. 3. **Präzisionstest:** Verwendung von `writeMicroseconds(1500)` zur exakten Zentrierung, gefolgt von der Überprüfung der Winkelgenauigkeit mittels eines Geodreiecks auf dem Rotorkopf.

6.10. Weiterführende Literatur

Towerpro SG90 Servo Motor and its application,Vayuyaan; Autor: Raj Sawkare; 03.12.2022 entnommen 19.04.2026
Servomotor (SG90); SunFounder Ultimate Sensor Kit; entnommen 19.04.2026

Bibliography

[1] TowerPro Ltd: *SG90 9g Micro Servo Datasheet & Specifications.*

[2] Arduino Documentation: *Servo Library Reference & Tutorials.*

[3] Components101 (2020): *SG90 Servo Motor Pinout, Datasheet & Wiring.*

[4] ARM mbed OS Documentation: *PWM Configuration for Servo Control on nRF52-based boards.*

Part III.

Domain Knowledge - Tools

7. Python Tool XY

8. Hardware Tool XY

Part IV.

Developmenmt

9. Flow Chart Development XY

Part V.

Monitoring

10. Monitoring XY

Part VI.

Verification - Evaluation - Conclusion

11. Evaluation/Verification XY

12. To DoX Y

13. Conclusion XY

Part VII.

Appendix

14. Bill of Material

15. SBOM

requirements.txt

16. Package Example

16.1. Introduction

16.2. Description

16.3. Installation

`pip packageExample`

16.4. Example - Manual

16.5. Example

16.6. Example - Code

16.7. Example - Files

`PackageExample.py`

16.8. Further Reading

References

[Aba+16] M. Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

Part VIII.

Hints - Examples for LaTeX - Read, Use and Remove

17. Hints

Please, use **`biber.exe`**.

If you want to create a symbol directory, call makeindex:
`makeindex %.nlo -s nomencl.ist`

17.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier curves are formed with the help of Bernstein polynomials. If at least two points and two tangents are known, control points can be determined with which a control polygon is formed. The course of the curve is oriented to this control polygon. The number of control points depends on the degree of the Bézier curve. A Bézier curve of degree n has n+1 control points. The Bézier curve is calculated using the De Casteljau algorithm.

Definition. In the interval [0; 1] the **Bernstein polynomial of nth degree** is defined by:

$$b_{i,n}(\lambda) = \binom{n}{i}(1-\lambda)^{n-i}\lambda^i, \quad \lambda \in [0,1], \quad i = 0, \ldots, n \tag{17.1}$$

Definition. The binomial coefficient is defined by:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \ldots, n \tag{17.2}$$

Here the Bernstein polynomials $b_{i,n}$ form a basis of the vector space $\mathbb{P}^n(I)$ of polynomials of degree at most n over I. Thus every polynomial of degree at most n can be written uniquely as a linear combination. [Far02]

Beispiel. Determination of Bernstein polynomials for $n = 3$ holds:

$$\begin{aligned}
b_{3,0}(\lambda) &= \binom{3}{0}(1-\lambda)^{3-0}\lambda^0 &&= (1-\lambda)^3 \\
b_{3,1}(\lambda) &= \binom{3}{1}(1-\lambda)^{3-1}\lambda^1 &&= 3\lambda(1-\lambda)^2 \\
b_{3,2}(\lambda) &= \binom{3}{2}(1-\lambda)^{3-2}\lambda^2 &&= 3\lambda^2(1-\lambda) \\
b_{3,3}(\lambda) &= \binom{3}{3}(1-\lambda)^{3-3}\lambda^3 &&= \lambda^3
\end{aligned}$$

$b_{3,i}(\lambda)$ with $i = 0, 1, 2, 3$ are the cubic Bernstein polynomials of degree 3.

Definition. Given are the points

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \ldots, n$$

A **Bézier curve** is then defined by

$$C(\lambda) = \sum_{i=0}^{n} Q_i \cdot b_{i,n}(\lambda) \tag{17.3}$$

The points $Q_i, i = 0, \ldots, n$ are called **control points**.

The control points of a Bézier curve form the so-called control polygon.

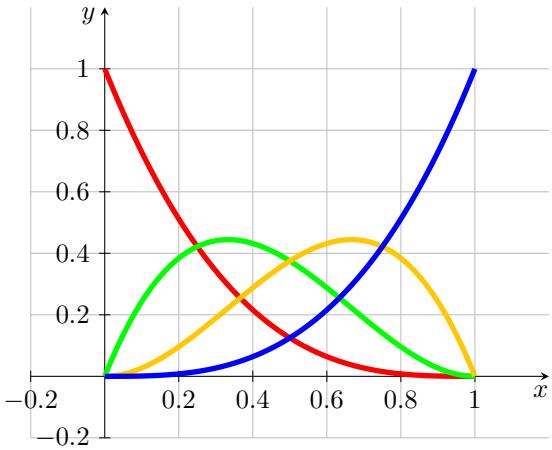


Figure 17.1.: Bernstein polynomial of degree 3

Bemerkung. Let the starting point P_0 and the end point P_1, as well as the tangents $\vec{t}_0$ and $\vec{t}_1$ be given. The tangents are not necessarily normalised.
The control points Q_0, Q_1, Q_2 and Q_3 of the associated Bézier curve are given by the following equations:

$$Q_0 = P_0, \qquad Q_1 = P_0 + \lambda_0 \vec{t}_0, \qquad Q_2 = P_1 - \lambda_1 \vec{t}_n, \qquad Q_3 = P_1 \tag{17.4}$$

[Jak+10]

Beispiel. Given are

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Then, according to theorem **??** for control points of the associated Bézier curve results:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \qquad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \qquad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

The Bézier curve and its control points are shown in the figure **??**.

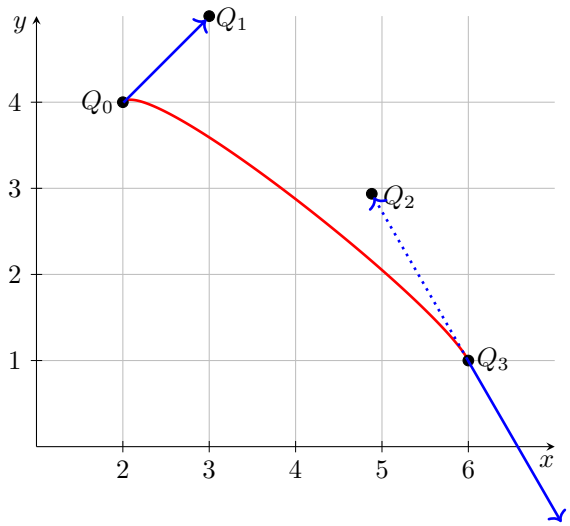


Figure 17.2.: Bézier curve for example 17.1

18. Verrundung mit einem Kreisbogen

18.1. Gleichungen

Blending with an arc is a simple variant of smoothing corners. This variant enables the processing and the generation of files according to DIN 66025. For smoothing, three points P_0 and S and P_1 are always considered, thus a symmetric Hermite problem results. According to theorem **??** it is assumed to be in the standard form $HP(L, \alpha)$.

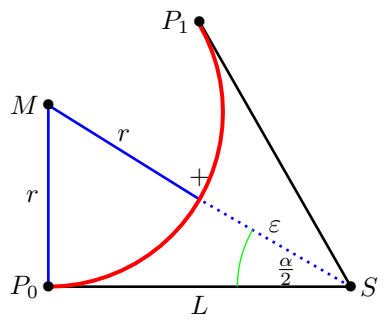


Figure 18.1.: Smoothing a corner with the help of an arc - triangle

The figure **??** represents the situation. The points P_0, S and P_1 are given. The included angle $\alpha = \angle(P_0; S; P_1)$ as well as the distance $L = \|\|S - P_0\|\| = \|\|P_1 - S\|\|$ are shown in the graph. The red arc is the desired result. The distance of its centre M to S is then $r + \varepsilon$, where r is the radius of the arc and ε is the given tolerance. Thus we obtain a right triangle P_0SM whose edge lengths are L, $r + \varepsilon$ and r.

According to the definition of the sine and the tangent, it follows:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r+\varepsilon} \qquad \text{und} \qquad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \quad \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \qquad \text{und} \qquad r = \cot\left(\frac{\alpha}{2}\right) L$$

The two equations can be combined to give the following conditions:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \qquad \text{bzw.} \qquad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

This gives the equation for the radius:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

The factor for converting L and ε can be simplified using trigonometric transformations.

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1-\sqrt{\frac{1-\cos(\alpha)}{2}}}{\sqrt{\frac{1+\cos(\alpha)}{2}}} = \frac{\sqrt{2}-\sqrt{1-\cos(\alpha)}}{\sqrt{1+\cos(\alpha)}} = \frac{\sqrt{1+\cos(\alpha)}-\sin(\alpha)}{1+\cos(\alpha)}$$

By comparing this with the symmetric Hermite problem in standard form, the following notation is then obtained:

$$\frac{1-\cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}}$$

The previous considerations are now summarised in the following sentence.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ uad $\alpha \in (-\pi;\, \pi]$.

Then an arc can be found that connects the points P_0 and P_1, has the same tangent directions at the two points.

For the circular arcs applies:

$$r = L\cdot\left|\frac{P_{1,x}}{P_{1,y}}\right|; \quad \phi_0 = -\operatorname{sign}(\alpha)\cdot\frac{\pi}{2}; \quad \phi_1-\phi_0 = \operatorname{sign}(\alpha)\cdot\pi-\alpha = \beta;$$

$$M = P_0 + \operatorname{sign}(\alpha)\cdot r\cdot\vec{t}_0^{\perp} = \operatorname{sign}(\alpha)\cdot r\cdot\begin{pmatrix}0\\1\end{pmatrix}$$

From the specification of the distance L, the maximum error can be calculated, as shown in theorem **??**. According to the derivation, it is also possible to specify the maximum error ε and determine the maximum distance L from it.

Satz. Let a symmetric Hermite problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ be given. Without restriction of generality, it is in the standard form

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}L+L\cdot\cos(\alpha)\\L\cdot\sin(\alpha)\end{pmatrix}, \begin{pmatrix}\cos(\alpha)\\\sin(\alpha)\end{pmatrix}, \begin{pmatrix}L\\0\end{pmatrix}, L\right\}$$

with $L \in \mathbb{R}^{>0}$ and $\alpha \in (-\pi;\, \pi]$.

a) Let the maximum error ε be given. The maximum distance L for which an arc exists according to the theorem **??** that takes the error into account is given by:

$$L(\varepsilon,\alpha) = \varepsilon\cdot\frac{P_{1,x}}{\sqrt{P_{1,x}}-P_{1,y}} = \varepsilon\cdot\frac{L+L\cdot\cos(\alpha)}{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}$$

b) When the distance L is specified, the following maximum error results:

$$\varepsilon(L,\alpha) = L\cdot\frac{\sqrt{P_{1,x}}-P_{1,y}}{P_{1,x}} = L\cdot\frac{\sqrt{L+L\cdot\cos(\alpha)}-L+L\cdot\cos(\alpha)}{L+L\cdot\cos(\alpha)}$$

These considerations result in limitations for the use of this strategy, which are summarised in the following comment.

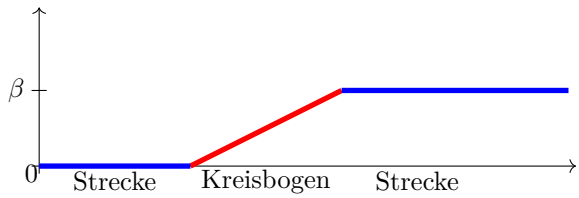


Figure 18.2.: Angular change with blending arc

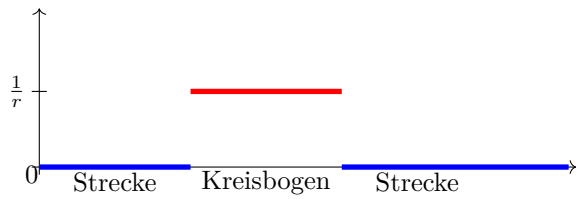


Figure 18.3.: Curvature progression for a blending arc

Bemerkung. The following conditions for applying the strategy of a circle must be fulfilled:

a)
$$P_0 \neq P_1$$

b)
$$\vec{t}_0 = -\vec{t}_1 \quad \Leftrightarrow \quad \alpha = 0$$

c)
$$\vec{t}_0 = \vec{t}_1 \quad \Leftrightarrow \quad \alpha = \pi$$

18.2. Bewertung

Rounding by means of a circular arc leads to a continuous course of the angle change. The figure **??** illustrates this.

Due to the rounding with a circular arc, the curvature of the curve is not continuous. The figure **??** shows that the curvature is constant in the range of distances 0 and on the circular arc with the value $\frac{1}{r}$, where r is the radius of the circular arc used. Due to the formula $a = \frac{v^2}{r}$ for a movement on a circular arc, the acceleration course exhibits continuity jumps at the transitions for a constant path velocity.

Depending on the angle change β at the corner S and the tolerance ε, the maximum length of the shortening of the lines can be calculated. The figure **??** shows the corresponding diagram, where the tolerance ε is set to the value 1.

The area provided can also be specified. Depending on the distance L from the corner point S, the deviation ε can be calculated. The figure **??** represents the function as a function of L and the angle α.
The figures **??** and **??** show the curves of the length as a function of the angle change for a fixed tolerance and the error as a function of the angle change for a given length. If the angle change is minimal, then the error or length L is extreme. In this case,

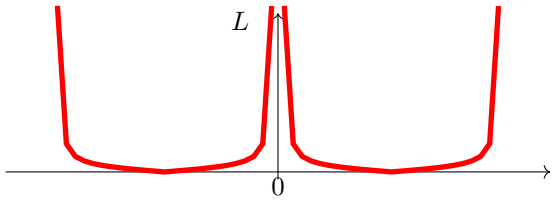


Figure 18.4.: Maximum range for a blending arc of a circle - $L(\varepsilon = const, \alpha)$

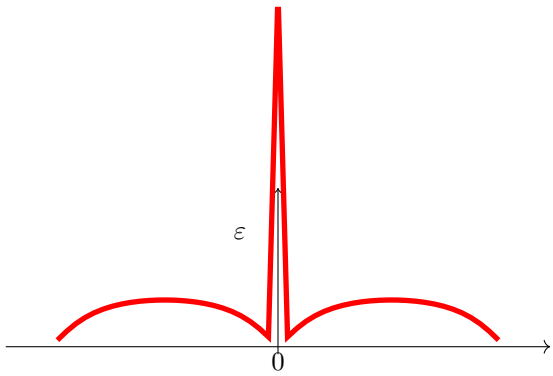


Figure 18.5.: Deviation when specifying the distance L when rounding with a circular arc

rounding by means of an arc is not possible. In order to develop a sensible strategy, a minimum angle change must be specified from which a blemnding arc is permitted. Likewise, a maximum angle change must also be defined. For an angular change of $\pm\pi$ is a bend. In this case, a blending is also not possible.

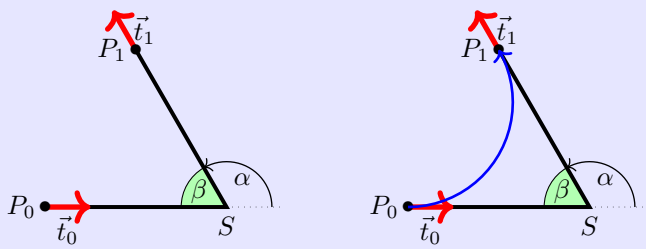
18.3. Examples

Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$

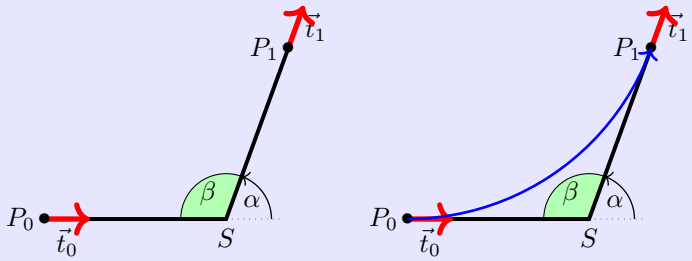


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$

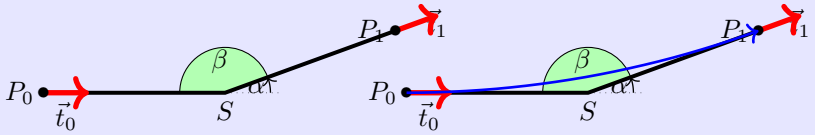


Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = \frac{1}{9}\pi$.
For the blending arc follows:

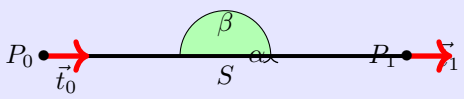
$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$



Beispiel. Let it be the symmetric Hermite problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

with $L = 6$ and $\alpha = 0$.
There is no blending arc here.

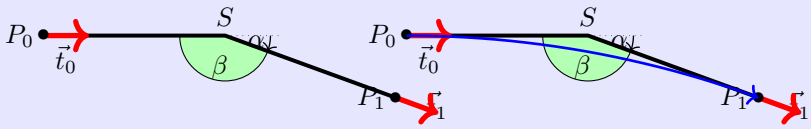


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{1}{9}\pi\right)\\6.0\cdot\sin\left(-\frac{1}{9}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{1}{9}\pi\right)\\\sin\left(-\frac{1}{9}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{1}{9}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-34,0277\end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

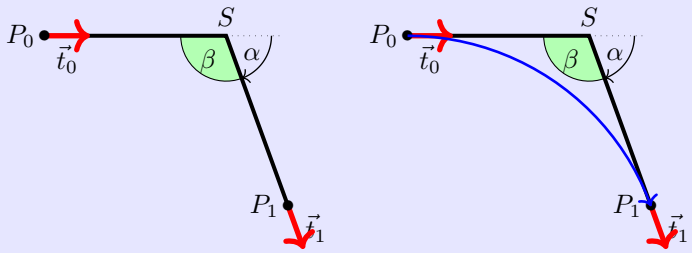


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{7}{18}\pi\right)\\6.0\cdot\sin\left(-\frac{7}{18}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{7}{18}\pi\right)\\\sin\left(-\frac{7}{18}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{7}{18}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-8,5689\end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$

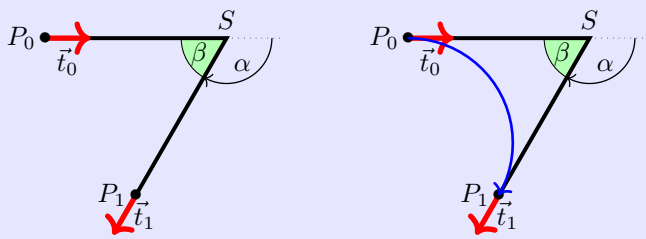


Beispiel. Let it be the symmetric Hermite problem

$$\left\{\begin{pmatrix}0\\0\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}6.0+6.0\cdot\cos\left(-\frac{2}{3}\pi\right)\\6.0\cdot\sin\left(-\frac{2}{3}\pi\right)\end{pmatrix}, \begin{pmatrix}\cos\left(-\frac{2}{3}\pi\right)\\\sin\left(-\frac{2}{3}\pi\right)\end{pmatrix}, \begin{pmatrix}6.0\\0\end{pmatrix}, 6.0\right\}$$

with $L = 6$ and $\alpha = -\frac{2}{3}\pi$.
For the blending arc follows:

$$M = \begin{pmatrix}0\\-3,4641\end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



19. Maple files

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

19.1. Geometries with Maple

The algorithms presented can be well represented using geometries. Two aspects can be investigated. On the one hand, the effort for an implementation, the computational accuracy and the stability of an algorithm are interesting; on the other hand, the methods are to be evaluated and compared with regard to their usefulness. For this purpose, modules for the formula manipulation system Maple [Wat17c] were developed. Maple offers the environment to implement algorithms easily and quickly. Furthermore, graphical representation is possible with simple means. However, a simple workbook of Maple is not designed for very large software projects. Here, one must resort to the possibility of creating and using libraries. On the one hand, Maple offers the possibility of creating one's own libraries, so-called modules. In this way, one remains within the environment and syntax of Maple. This path will be pursued further here. Another possibility is the use of libraries created by means of a higher programming language, e.g. C++, the so-called DLLs.
In the following, the creation and use of a test environment with the help of modules is described. First, the geometry elements that are stored in various modules and their use are described. The structure and use of a module in Maple is then explained so that own extensions and additions are possible.

19.2. Geometry elements used

This is a test environment for the algorithms presented, only geometries that have been described are also used.

- points (`MPoint`)
- lines (`MLine`)
- arcs (`MArc`)
- Bézier curves (`Bezier`)
- polygon courses (`MPolygon`)
- Geometry List (`MGeoList`)
- Hermite problems (`MHermiteProblem`)
- Symmetric Hermite Problems (`MHermiteProblemSym`)

For each geometry element a corresponding module has been created, the name of which is given in the list above. The list of which functions are available is presented in another section.
When implementing such a project, it quickly becomes clear that when using several geometry elements, a clear data structure and well-defined access to the data is essential. For example, when representing straight lines, the decision must be made

whether the representation should be point-directional or by means of two points. The choice is generally made depending on the application. Here, the path is taken that, due to a well-defined access to the data, the use of both representations is possible.

19.3. Building the data structure

The idea is to use a data structure that is as uniform and simple as possible. Maple does offer the possibility to define objects. However, it is difficult to use within a procedural environment. Therefore, all data is basically represented as lists. The first list element always contains an identifier of the element **??**. This is followed by the data, which in turn can be geometry elements. It should be noted that direct access to the data cannot be prevented; here the user is responsible.

Access to the data should be exclusively via procedures of a module. The following is an example of the data structure for points:

```
[MVPOINT,[x,y]]
```

The creation of a point then takes place via a procedure `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

When called up in the test file, the following is then output:

```
TestP0 := ["Point",[10,15]]
```

These data structures are used for the calculation of geometries. They are used in functions or procedures. Unlike variables, the data structures and procedures are defined globally and not locally. Under the command `export` the procedures are declared at the beginning of the module and can thus also be used outside the module. If, for example, a data structure from `MPoint` is to be used in another module or outside the modules, it is called as follows:

```
P0 := MPoint:-New(x,y);
```

In addition to the modules for the geometries, there is a module (MConstant) for saving constants and names. There, for `MVPOINT`, for example. „Point “ or e.g. a constant for the comparison to zero for real numbers.
Finally there is the test file, which is not a module, in which the modules are loaded, called and tested in their function. More about this file later in „1.4 Maple test file “.

19.4. Structure of a module

The following section deals with the purpose of modules and their structure.

The project is about capturing the above geometries in a data structure and representing them in Maple. However, the calculations and formulas that have to be used are a hindrance to the final representation. Modules are very well suited for summarising and hiding the calculations that take place in functions. This way, the important functions can be accessed in Maple without seeing what is in them.

Example:
The function `Angle` from the module `MPoint`: Function to calculate an angle between location vector and x-axis:

```
Angle := proc(P)
   local alpha, x, y;
```

```
   x := GetX(P);
   y := GetY(P);
   alpha := 0;
   if abs(x) < 0.00001
      then
         alpha := 3*Pi/2;
      else
         alpha := Pi/2;
      end if;
   else
      alpha := arctan(y/x);
      if x < 0
      then
         alpha := alpha + Pi;
      else
         if y < 0
         then
            alpha := alpha + 2*Pi;
         end if;
      end if;
   end if;
   return factor(alpha);
end proc;
```

This function is long, but it only represents a small part of the programme for the representation in question. That is why it is in the module and can thus be called externally with a single command:

```
Winkel := MPoint:-Angle(P)
```

The following section describes the structure of a module. Since Maple offers a wide range of possibilities, a restriction is made. Only the structure of the modules used is described, here on the basis of the module`MPoint`. For more detailed possibilities, please refer to the Maple manual [Wat17c].

<u>structure:</u>

The module must first be started. This is done with the name (here always a capital M and the geometry) of the module and the following command:

```
MPoint := module()
```

Then, similar to procedures, the variables and functions must be defined. You can declare them either locally or globally. If the variables or procedures are only used and changed in the module, the declaration is made with the command `local` as follows:

```
 local  Variable names, with, comma, separated;
```

If the variables or procedures are also to be usable outside the module, this is defined with `export`:

```
export Function names, with, comma, separated;
```

This is followed by the specification of the options:
`option package;`,
the description of the module:

```
description “Self-selected module description, e.g. module for points
“;
```

and the initialisation of the module:

```
ModuleLoad := proc
  MVPOINT:=MConstant:-GetPoint();
  print("Module MPoint is loaded");
end proc;
```

From here on, programming is done as usual in Maple. The previously defined procedure and variable names are used and programming is done with commands that are also used outside of a module in Maple. It is important to know that with modules, each function can be called constantly, so the order of the functions is irrelevant.

To finally end the module, use the following command:

```
end module;
```

19.4.1. General structure of a module

In summary, the modules have the following structure:

```
Modulname := module()

   local Names, with, comma, separated;

   export Names, with, comma, separated;

   global Names, with, comma, separated;[1]

   option package;

   description „ Self-selected module description“;


   ModuleLoad := proc()[2]
      MVPOINT:=MConstant:-GetPoint();
      print("Modul MPoint is loaded");
   end proc;

 Procedure1 := proc (Passing parameters)
      inhalt;
   end proc;

   Procedure2 := proc (Passing parameters)
      content;
   end proc;


   ...

end module;
```

[1] Possible, but not used in the modules
[2] Example `MPoint`

19.4.2. Saving a module

The management of modules is done automatically by Maple. However, since the modules are passed on and have to be edited individually, some settings have to be made. Therefore, the following prefix is used for each Maple file of the project:

```
restart;
with(LibraryTools);
lib := "C:/FH/Tools/Maple/MyLibs/Blending.mla";
march('open', lib);
ThisModule := 'MArc';
```

The first line initialises the system. The next 3 lines enable working with archives. In the second line, the tools are loaded so that the variable `lib` can be occupied. All modules are stored in an archive; in this example it is the file `Blending.mla` in the directory `C:/FH/Tools/Maple/MyLibs/`. The command `ThisModule := 'MArc';` contains the name of the current module.

A module is then saved using the command `savelib('ModuleName')`. A file `ModuleName.mla` is then created. Depending on the configuration of Maple, the set path where the file is automatically created may not be writable. Then you can configure the system so that the mla file is stored in the current directory or in a directory of your choice.

If the above prefix is used for a Maple file, all that is required to save the module is
`savelib(ThisModule, lib);`

19.4.3. Verwendung eines Moduls

A module that has been saved can now be used in other Maple worksheets. To do this, the command
`with(ModuleName)`
is used. If you have used a special path, Maple cannot find the file `ModuleName.mla`. Then the path must be made known, e.g.:
`savelibname := "c:/Maple/MyLibs";", savelibname;`
Now the procedures of the module can be accessed. An overview of all exported procedures is provided by the command
`Describe(ModuleName)`
is displayed. A list of the names including the names of the transfer parameters is displayed. If a procedure is equipped with the field `description`, this text is also displayed.

19.4.4. Creating a Maple module

Creating your own module is done quickly if you use the framework above. However, one should make some considerations beforehand and maintain a standard.

Before creating an own module, the data model and the corresponding procedures should be worked out. A programme flow chart is useful here. The central task of the module is usually quickly determined. In addition, however, the following rules should be observed.

Rule 1 Basically, both the module and each procedure receive a description. This is not limited to the optional argument `description`, but is placed in front of each procedure. The description basically contains the description of the task. The prerequisite is also mentioned or an error handling is described. Then follows the description of all input parameters, their function and their data structure. The return value is then described.

rule 2 Each module receives a procedure `Version()`, which returns the current version number.

rule 3 Data is not global. To access data, procedures `Set*` and `Get*` are provided. These procedures are also used within the procedures of a module.

rule 4 At least one test function is written for each procedure. The test function can be used to illustrate the use and any special features.

19.5. Functions of the modules

In the following, the individual modules are listed. The data structure of each module and all functions with associated tasks are listed.

19.5.1. Module `MPoint`

`MPoint` is a module for points and works with a data structure and with procedures/functions. The data structure `New` for a point is a list, which is structured as follows:

```
[MVPOINT, [x,y]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Point" and the elements are the x and y coordinates for a point. No distinction is made here between point and vector. A point can also be seen as a vector from the coordinate origin to the point.
Via the Get functions `GetX` and `GetY` one can capture the coordinates of the point and use them in other functions. The other functions can then be used to calculate with the points/vectors.

MPoint		
E	**New**	Data structure for a point
E	**GetX**	Reading the x-coordinate
E	**GetY**	reading the y-coordinate
E	**Angle**	calculating the angle between the x-axis and the point
E	**Add**	Calculates a linear combination of two points/vectors
E	**Sub**	Calculates the difference of two points/vectors
E	**Cos**	Calculates the cosine between two vectors
E	**Sin**	Calculates the sine between two vectors
E	**Scale**	Scales a vector with a factor
E	**Perp**	Calculates a vector that is orthogonal to the given vector
L	**IllustrateXY**	Plot function to illustrate a blue point
E	**Illustrate**	Plot of a blue point
E	**Plot**	Plot of a green point
E	**Plot2D**	Plot of a point with own options
E	**Length**	Calculates the distance of the point to the coordinate origin
E	**Uniform**	Normalises the vector to length 1
E	**LinetoVector**	Calculates vector from a distance
E	**Distance**	Calculates the distance between two points

19.5.2. Module MLine

MLine is a module for lines and works with a data structure and with procedures/functions. The data structure New for a line is a list which is structured as follows:

```
[MVLINE, [P0,P1]]
```

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „Line“ and the elements are the start and end points for a line.
The data structure NewPointVerctor is also a data structure for a line, but here the line is defined by a start point and a direction vector.
The Get functions StartPoint and EndPoint can be used to capture the start and end points of the route and use them in other functions. The other functions can then be used to calculate with the routes.

MLine		
E	**New**	Data structure for a line (two-point form)
E	**NewPointVector**	creation of a route (point-direction form)
E	**StartPoint**	reading the starting point
E	**EndPoint**	reading the end point
E	**Position**	Calculate a point that lies on the line
E	**Plot2D**	Plot a part of the route starting from the starting point
E	**Plot2DTangent**	Plot of a point with tangent
E	**Plot2DTangentArrow**	Plot of a point with tangent (as arrow)
E	**LineLine**	Calculation of the intersection of two straight lines.
E	**AngleLine**	Calculation of the angle between two straight lines

19.5.3. Module MArc

MArc is a module for circular arcs and works with a data structure and with procedures/functions. The data structure New for an arc is a list that is structured as follows:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Arc" and the elements are the x- and y-coordinate for the centre, the radius, the start angle and the angle change for an arc.
The Get functions GetM, GetMX, GetMY, GetR, GetPhi, and GetAlpha can be used to collect the data for an arc and use it in other functions. The other functions can then be used to calculate with the arcs.

MArc		
E	**New**	Data structure for an arc
E	**GetMX**	Reading the x-coordinate of the centre point.
E	**GetMY**	read the y-coordinate of the centre point
E	**GetR**	read the radius
E	**GetPhi**	reading the start angle
E	**GetAlpha**	Reading the change of angle
E	**GetM**	reading the centre point
E	**Position**	Calculating a point on the arc
E	**Plot2D**	Plot a part of the arc starting from the start angle
E	**Blend**	calculating an arc from a symmetric Hermite problem

19.5.4. module MBezier

The New data structure for a polygon is a list constructed as follows:

[MVBEZIER, [PointList]]

Within the module MBezier exist the procedures listed in the following table.

MBezier		
E	**New**	Manual input of control points for a Bézier curve
E	**Version**	Output of the verions
E	**BlendCurvature**	Determination of control points from symmetric Hermite problem
E	**BlendCurvatureEpsilon**	determination of control points from symmetric Hermite problem with given error
E	**Position**	position on the Bézier curve
E	**GetTheta**	Reading the angle
E	**GetEpsilon**	reading the tolerance
E	**GetControlPoint**	Read the control points
E	**Plot2D**	Read the Bézier curve
E	**PlotControlPoints**	representation of all control points

19.5.5. Module MPolygon

MPolygon is a module for polygons and works with a data structure and with procedures/functions. The data structure New for a polygon is a list that is structured as follows:

[MVPOLYGON, [PointList]]

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is "Polygon" and the elements are any number of points.
Using the Get functions GetPoint and GetN you can get the number of points and the points themselves and use them in other functions. The other functions can then be used to calculate with the points or the polygon course.

MPolygon		
E	**New**	Data structure for a list of points
E	**GetPoint**	Reading the ith point from the point list
E	**GetN**	Determine the number of points in the point list
E	**Length**	Determination of the Euclidean length of the polygon
E	**Position**	Calculation of a point on the polygon course
E	**Tangents**	calculation of the tangent
L	**Plot2DAll**	Plot list of all points
E	**Plot2D**	Plot of all points as polygonal plots.
E	**Plot2DTangent**	representation of the polygon course with tangent
E	**PlotPoints**	representation of all points

19.5.6. module MGeoList

MeoList is a module for a geometry list and works with a data structure and with procedures/functions. The data structure New for a geometry list is a list that is structured as follows:

[MVGEOLIST, []]

Its first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „GeoList“ and the elements are any number of individual geometries.
Using the Get functions `GeoGeo` and `GetN`, the i-th element of the list and the number of elements in the list can be captured and used in other functions. The other functions can then be used to calculate with the geometries or the list. In the functions `Length`, `Plot2DAll`, `Plot2D` and `Position` the functions are called in themselves. This is possible because the functions work independently of each other.

MGeoList		
E	**New**	Data structure for a geometry list
E	**Append**	Append a geometry element to the data structure
E	**Prepend**	Inserting a geometry element as the first element of the list
E	**Replace**	Replace a geometry element with another one.
E	**GetN**	Determine the number of geometry elements in the list
E	**GeoGeo**	Reading the i-th geometry element
E	**Length**	Calculate the Euclidean length of the geometry elements
E	**Position**	Calculates a point on the geometry
L	**Plot2DAll**	Plot function for plotting the geometry elements
E	**Plot2D**	Plot a part of the geometry list starting from the first element

19.5.7. Module MHermiteProblem

`MHermiteProblem` is a module for Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a route is a list, which is structured as follows:

```
[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „HermiteProb“ and the elements are two points with associated tangents (lines).
Using the Get functions `StartPoint`, `EndPoint`, `StartTangent` and `EndTangent`, the points and their tangents can be captured and used in other functions. The other functions can then be used to calculate with the data for the Hermite problem.

MHermiteProblem		
E	**New**	Data structure for a Hermite problem
E	**StartPoint**	reading the start point
E	**EndPoint**	Reading the end point command
E	**StartTangent**	reading the start tangent
E	**EndTangent**	Reading the end tangent
E	**Plot2D**	Plotting the Hermite problem

19.5.8. Module MHermiteProblemSym

`MHermiteProblemSym` is a module for symmetric Hermite problems and works with a data structure and with procedures/functions. The data structure `New` for a symmetric Hermite problem is a list built as follows:

```
[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]
```

Your first element is a name to identify the module. Its second element is again a list containing the elements of the geometry. In this case the name is „SymHermiteProb“

and the elements are two points with associated tangents, their intersection, and the distance of the points to the intersection.

Via the Get functions `StartPoint`, `EndPoint`, `StartTangent`, `EndTangent`, `CrossPoint` and `ParameterL` one can acquire the data and use it in other functions. The other functions can then be used to calculate with the data for the symmetric Hermite problem.

MHermiteProblemSym		
E	**New**	Data structure for a symmetric Hermite problem
E	**StartPoint**	reading the start point
E	**EndPoint**	Reading the end point command
E	**StartTangent**	reading the start tangent
E	**EndTangent**	Reading the end tangent
E	**ParameterL**	reading of the distance
E	**CrossPoint**	reading of the intersection point
E	**Plot2D**	Plotting of the symmetrical Hermite problem
E	**Create**	creation of a Sym. Hermite problem by 3 points
E	**BlendArc**	rounding of the corner point by an arc

19.5.9. Module MConstant

`MConstant` is a module for storing fixed constants and names to identify data structures. In the locally declared functions, starting with CV, the constants for declaring the different geometries are defined. These are names for recognising the geometry elements in the test file. In the globally declared functions, starting with Get, the names for identifying the geometry elements are returned.

MConstant		
L	**NULLEPS**	constant to compare to zero
L	**CVPOINT**	constant for geometry elements: Point
L	**CVLINE**	constant for geometry elements: Line
L	**CVARC**	constant for geometry elements: Arc
L	**CVPOLYGON**	constant for geometry elements: polygon
L	**CVGEOLIST**	constant for geometry elements: GeoList
L	**CVHERMITEPROBLEM**	constant for geometry elements: HermiteProb
L	**CVHERMITEPROBLEMSYMMETRIC**	Const. for geometry elements: SymHermiteProb
L	**CVBIARC**	Const. for geometry elements: Biarc
E	**GetNullEps**	return of the zero comparison command.
E	**GetPoint**	identifier for points
E	**GetLine**	Identifier for lines
E	**GetArc**	identifier for circular arcs
E	**GetPolygon**	identifier for polygons
E	**GetGeoList**	Identifier for geometry lists
E	**GetHermiteProblemSymmetric**	Identifier for symmetric Hermite problems
E	**GetHermiteProblem**	ID for Hermite problems
E	**GetBiarc**	identifier for Biarcs

19.5.10. Module MGeneralMath

`MGeneralMath` is a module for general mathematical functions and works with the data structures and procedures.

MeneralMath		
E	MPoint	Data structure for a point
E	MPointX	Reading of the x-coordinate for a point
E	MPointY	reading the y-coordinate for one point
L	MPointIllustrateXY	plot structure for a blue point
E	MPointIllustrate	plot structure for a blue point
E	MPointPlot	Illustration of a green point
E	MLine	Data structure for a line
E	MLineStartPoint	Reading of the start point for a draw frame
E	MLineEndPoint	Reading the end point for a line
E	MPointOnLine	Calculation of a point on the line
E	MLinePlot2D	Plot the part of a line starting at the starting point
. E	MLineLine	calculating the intersection of two linesline

19.5.11. Module Biarc

Data Structure

Requirements and specifications:

The Biarc is to be used to solve a Hermite problem. A Hermite problem is defined as follows:

Given two points P_0 and P_1 with associated normalised tangents $\vec{t_0}$ and $\vec{t_1}$. The biarc must connect the points such that the tangents of the start and end points of the biarc coincide with the tangents of the Hermites problem.

Data structure:

The data structure `New` for a biarc must contain two arcs.

So the data structure for one arc is needed: `MArc:-New`.

This in turn contains the coordinates for the centre, the radius, the start angle and the angle change.

19.5.12. Module MBiarc

`MBiarc` is a module for Biarcs and works with a data structure and with procedures/functions. The data structure `New` for a Biarc is a list which is structured as follows:

```
[MVBIARC, [Arc0, Arc1]]
```

Its first element is a name to identify the module. Your second element is again a list containing the elements of the geometry. In this case the name is „Biarc“ and the elements are two arcs.

Using the Get functions `GetArc0` and `GetArc1` one can capture the two arcs for the biarc. The functions listed below can be used to calculate the data for the biarc.

MBiarc		
E	New	Data structure for a biarc
E	GetArc0	Reading of the first arc
E	GetArc1	Reading the second arc
E	Circle	calculating the circle K_J from the Hermite problem data.
E	Plot2DCircle	plot of the circle K_J
E	angle	Calculate the angle from the centre of the circle to points on the circle
E	Plot2D	Plot of the biarc
E	ConnectionPoint	Calculation of the connection point J (Equal Chord)
E	TangentTj	Calculation of the tangent to J
E	Tangent Biarc	rotation of Tj for the biarc.
E	BiarcCenter	Calculate the centres of the arcs of the biarc
E	Biarc	calculate the centre of the biarc.
E	Position	Determine a point on the biarc
E	Blend	Calculate the biarc only from the Hermite problem

19.6. Programme Flowchart

19.6.1. Overall Flow

19.6.2. Verrundung der Kurve

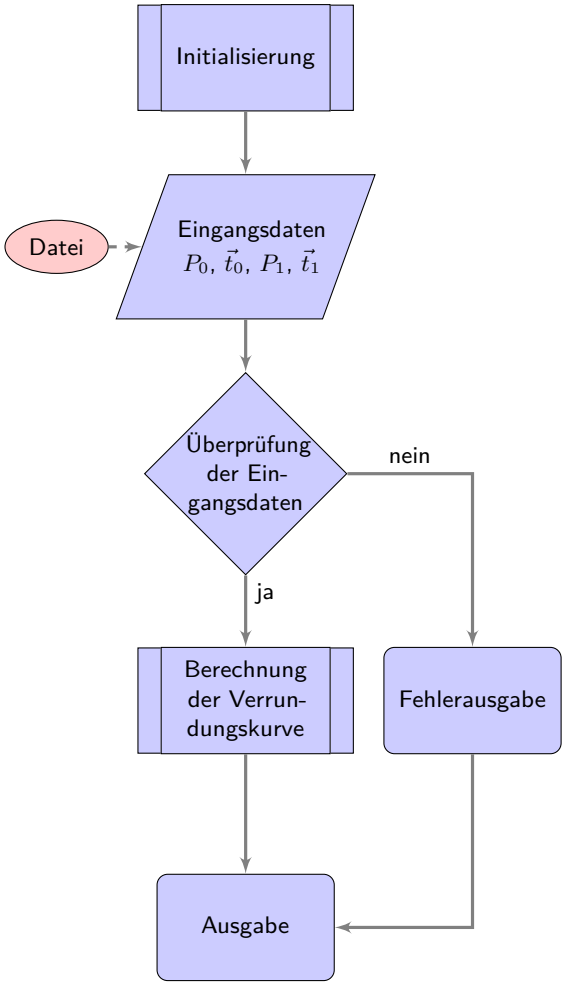


Figure 19.1.: Programme flow chart „Corner rounding“

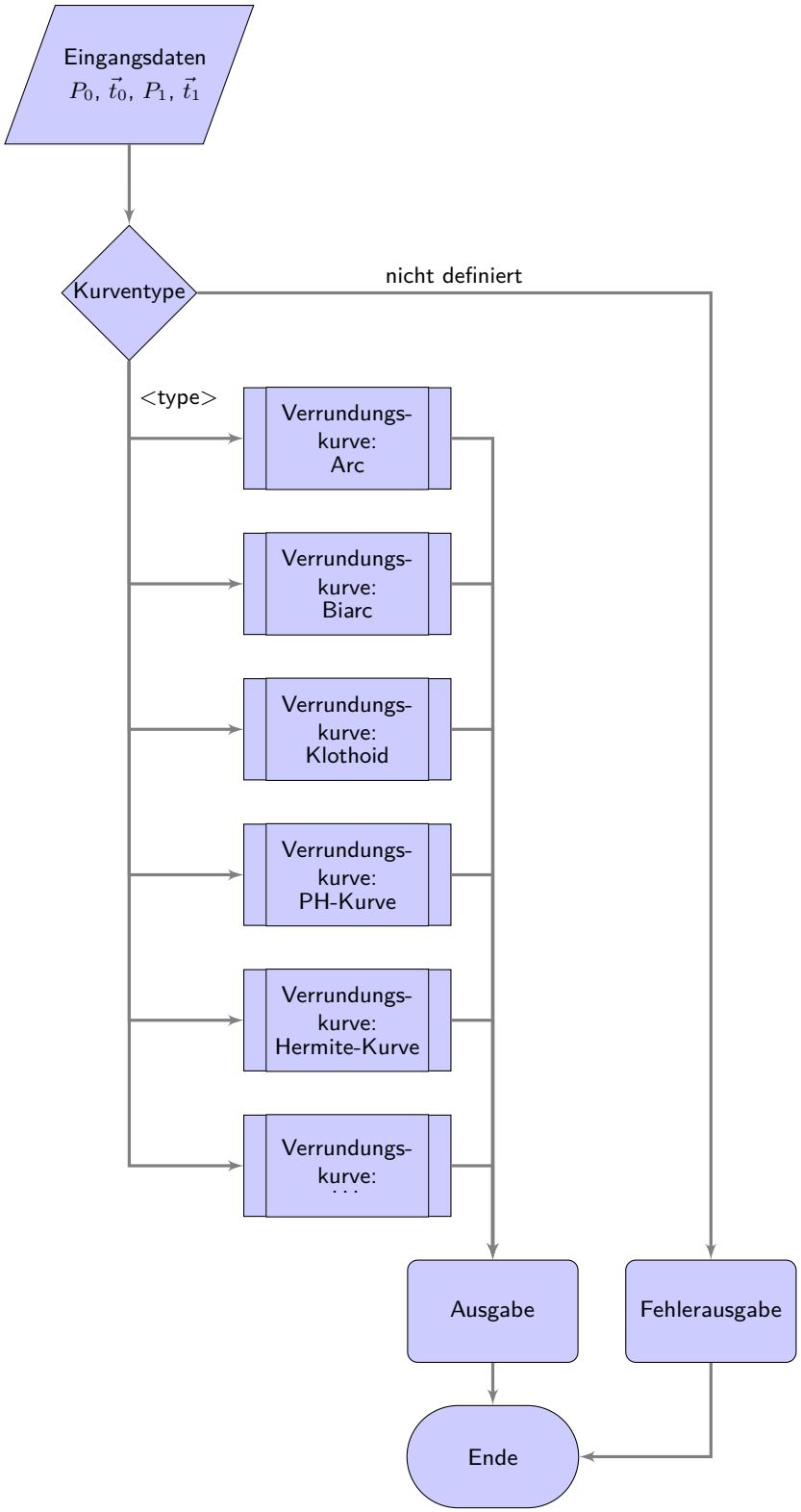


Figure 19.2.: Programme flowchart „Selection of the rounding strategies“

20. Representation of Python Programs

It is possible to integrate a part into the document, see 20.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 20.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file Blink.py.

```
%%
%% This is file `rename-to-empty-base.tex',
%% generated with the docstrip utility.
%%
%% The original source files were:
%%
%% fileerr.dtx  (with options: `return')
%%
%% This is a generated file.
%%
%% The source is maintained by the LaTeX Project team and bug
%% reports for it can be opened at https://latex-project.org/bugs/
%% (but please observe conditions on bug reports sent to that address!)
%%
%%
%% Copyright (C) 1993-2025
%% The LaTeX Project and any individual authors listed elsewhere
%% in this file.
%%
%% This file was generated from file(s) of the Standard LaTeX `Tools
     Bundle'.
%%
%%
%% It may be distributed and/or modified under the
%% conditions of the LaTeX Project Public License, either version 1.3c
%% of this license or (at your option) any later version.
%% The latest version of this license is in
%%    https://www.latex-project.org/lppl.txt
%% and version 1.3c or later is part of all distributions of LaTeX
%% version 2005/12/01 or later.
%%
%% This file may only be distributed together with a copy of the LaTeX
%% `Tools Bundle'. You may however distribute the LaTeX `Tools Bundle'
%% without such generated files.
%%
%% The list of all files belonging to the LaTeX `Tools Bundle' is
%% given in the file `manifest.txt'.
%%
 \message{File ignored}
\endinput
%%
%% End of file `rename-to-empty-base.tex'.
```

../../Code/HelloWorld/Blink.py

21. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 21.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 21.1.: The program “Hello World” for an Arduino microcontroller boardis inserted from the file Blink.ino.

```
/**
*
*   @file Blink.ino
*
*   @brief Simple program for testing the configuration
*
*
*   Turns an LED on for one second, then off for one second, repeatedly.
*
*   Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
*   it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
*   the correct LED pin independent of which board is used.
*   If you want to know what pin the on-board LED is connected to on your Arduino
*   model, check the Technical Specs of your board at:
*
*   https://www.arduino.cc/en/Main/Products
*
*   modified 8 May 2014
*   by Scott Fitzgerald
*   modified 2 Sep 2016
*   by Arturo Guadalupi
*   modified 8 Sep 2016
*   by Colby Newman
*
*   This example code is in the public domain.
*
*   https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
*/



/**
*   @brief the setup function runs once when you press reset or power the board
*
*   standard function of Arduino sketches
*
*   Initialization of the pin LED_BUILTIN as output
*
*   @param ---
*
*   @return void
*/
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

/**
*   @brief the loop function runs over and over again forever
*
*   standard function of Arduino sketches
*
*   swichting the led on / off for 1sec.
*
*   @param ---
*
*   @return void
*/

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

../../Code/Arduino/Blink/Blink.ino

22. First Chapter

...
A Speicherprogrammierbare Steuerung (SPS) is ...

A Computerized Numerical Control (CNC) needs a SPS (PLC) to ...

23. CAGD

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like "Huardest gefburn"? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.
The book CAGD by Gerald Farin is a classic on splines. [Far02]

The standard 66025 for programming CNC machines is also a classic; however, it does not deal with splines. [DIN66025-2]

Mr. F. Farouki has dealt with both CNC machine programming and splines. His article[1] on a real-time interpolator also shows this. [FS17]

A new dimension of machine tools have emerged with the invention of 3D printers. [Rus+07]

Another aspect of automation technology is communication. Another milestone has been reached with 5G technology. another milestone has been reached.[2]

[1] co-author is J. Srinathu
[2] Zaf+20.

A. drawings with tikz

The package tikz is a powerful tool for creating graphics. Many introductions exist. Here only the first steps are shown, so that you can easily create flowcharts.

Drawing a line and arrows

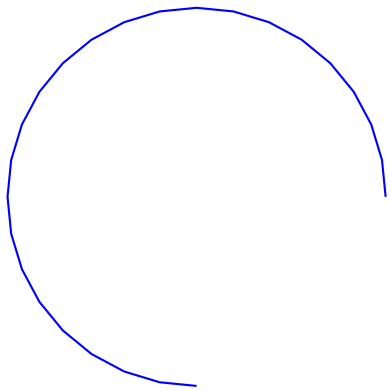
```
\begin{tikzpicture}
    \draw (0,0) -- (1,0);

    \draw[->] (2,0) -- (3,0);

    \draw[<->] (5,0) -- (6,0);
\end{tikzpicture}
```


Drawing a thick blue line

```
\begin{tikzpicture}
    \draw[line width=2pt, blue] (0,0) -- (1,0);

    \draw[line width=2pt, red, dotted] (2,0) -- (3,0);

    \draw[line width=2pt, dashed, green] (4,0) -- (5,0);

    \draw [thick,dash dot] (0,1) -- (5,1);
    \draw [thick,dash pattern={on 7pt off 2pt on 1pt off 3pt}] (0,2) -- (5,2);
\end{tikzpicture}
```

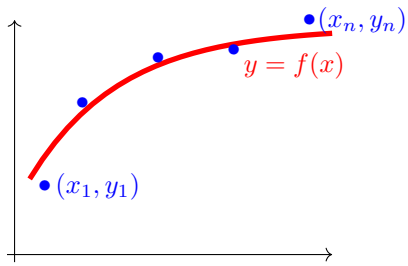
Drawing an arc

```
\begin{tikzpicture}
  \draw [blue,thick,domain=0:270] plot ({5+2.5*cos(\x)}, {1+2.5*sin(\x)});
\end{tikzpicture}
```



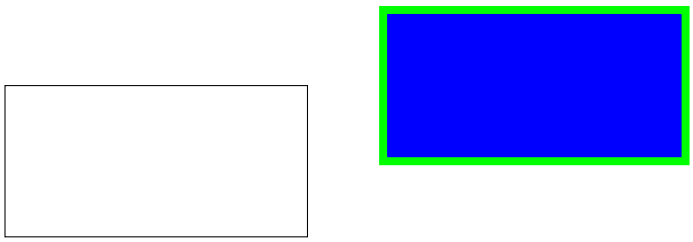
Draw a function

```
\begin{tikzpicture}[
    declare function={%
        F(\x)                    =3-2*pow(2.7979,-0.8*\x);
    }
 ]


    % Zeichnen der Funktion
    \draw[red,line width=2pt,domain=0:4] plot({\x},{F(\x)});

    % Bezeichnung
    \node[red] (O) at(3.5,2.5) {$y=f(x)$};

    % Punkte
    \node[blue] (P1n) at (0.9,0.9) {$(x_1,y_1)$};
    \node[blue] (P1) at (0.2,0.9) {$\bullet$};

   \node[blue] (P2) at (2.7,2.7) {$\bullet$};
   \node[blue] (P3) at (1.7,2.6) {$\bullet$};
   \node[blue] (P4) at (0.7,2) {$\bullet$};

   \node[blue] (P5n) at (4.4,3.1) {$(x_n,y_n)$};
   \node[blue] (P5) at (3.7,3.1) {$\bullet$};

   % Koordinatensystem
   \draw[color=black,->] (-0.2,-0.1) -- (-0.2,3.1);
   \draw[color=black,->] (-0.3,0) -- (4,0);
\end{tikzpicture}
```



Drawing rectangles and moving objects

```
\begin{tikzpicture}
    \draw (0,0) -- (4,0) -- (4,2) -- (0,2) -- cycle;

    \begin{scope}[shift={(5,1)}]
      \draw[green, fill=blue, line width=3pt] (0,0) -- (4,0) -- (4,2) -- (0,2) --
    \end{scope}
\end{tikzpicture}
```

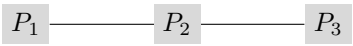


Use of variables

```
\begin{tikzpicture}
\pgfmathsetmacro{\PHI}{-15}
% Now use \PHI anywhere you want -15 to appear,
% can also be used in calculations like 2*\PHI
\def\x{10};

\draw[red] (0,4) -- (1-\PHI*0.5,4);
\draw[green] (0,2) -- (1+1/\x,2);
\draw[blue] (0,0) -- ({1+3*(\x/5+1)},0);
\end{tikzpicture}

\bigskip

%\usetikzlibrary{math} %needed tikz library

\begin{tikzpicture}
  \pgfmathsetmacro{\drehpunktx}{11.63815573}
  %Variables must be declared in a tikzmath environment but
  % can be used outside
  \tikzmath{\x1 = 1; \y1 =1;
  %computations are also possible
  \x2 = \x1 + 1; \y2 =\y1 +3; }
  \draw[->] (\x1, \y1)--(\x2, \y2);
\end{tikzpicture}
```

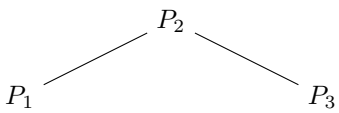
Use of points

```
%\usetikzlibrary{backgrounds}  % is needed

  \begin{tikzpicture}
    \node [fill=gray!30] (P1) at (0,0) { $P_1$ };
    \node [fill=gray!30] (P2) at (2,0) { $P_2$ };
    \node [fill=gray!30] (P3) at (4,0) { $P_3$ };

    \begin{scope}[on background layer]
        \draw (P1) -- (P3);
    \end{scope}
  \end{tikzpicture}
```



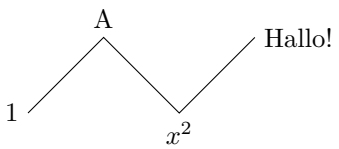
Use of nodes

```
  \begin{tikzpicture}
    \node  (P1) at (0,0){$P_1$};
    \node  (P2) at (2,1){$P_2$};
    \node  (P3) at (4,0){$P_3$};

    \begin{scope}
        \draw (P1) -- (P2) -- (P3);
    \end{scope}
\end{tikzpicture}
```



Use of nodes

```
\begin{tikzpicture}
  \draw (0, 0) node[left]{1}
        -- ++(1, 1) node[above] {A}
        -- ++(1,-1) node[below] {$x^2$}
        -- ++(1, 1) node[right] {Hallo!};
\end{tikzpicture}
```



B. Criteria for a good LaTeX project

A good report is not only characterised by good content, but also fulfils formal aspects. The following list should help to comply with basic rules. Before submitting, all points should be checked and ticked off.

- ☐ Is a suitable directory structure used?
- ☐ Is an appropriate division into files used?
- ☐ Are meaningful directory and file names used?
  - ☐ Are directory and file names such as report or term paper avoided?
  - ☐ Are meaningful names used for images and not „image1“?
  - ☐ Are directory and file names not too long?
  - ☐ Are special characters and spaces avoided?
- ☐ Are commands like \newline and \\ avoided?
- ☐ Are the LaTeXfiles clearly laid out?
  - ☐ Are indentations used in the LaTeXfiles?
  - ☐ Are free lines inserted?
  - ☐ Is the project mentioned in the header of the files?
  - ☐ Does the header of the files mention the main sources?
  - ☐ Is the author mentioned in the header of the files?
- ☐ Are all necessary files given?
- ☐ Are the temporary files deleted?
- ☐ Are graphics created by yourself?
- ☐ Are the sources of the images given?
- ☐ Are citations made with the command \cite?
- ☐ Is a bib file used?
- ☐ Are the entries of the bib-file clearly arranged?
- ☐ Are the entries of the bib-file edited to show them correctly?
- ☐ Are the correct types used for the bib entries?
- ☐ Are meaningful keys used in the bib files?

Unfortunately, the list cannot be complete, but it provides some clues.

Literature

[Aba+16] M. Abadi et al. "TensorFlow: A System for Large-Scale Machine Learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 265–283. ISBN: 978-1-931971-33-1. URL: https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

[DIN66025-2] DIN Deutsches Institut für Normung e.V. *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin, Sept. 1988.

[FS17] R. Farouki and J. Srinathu. "A real-time CNC interpolator algorithm for trimming and filling planar offset curves". In: *Computer-Aided Design* 86 (Jan. 2017). DOI: 10.1016/j.cad.2017.01.001.

[Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.

[Jak+10] G. Jaklic et al. "On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves". In: *Mathematics of Computation* (2010). [pdf].

[Rus+07] D. Russell et al. *Apparatus and Methods for 3D Printing*. United States Patent, Patent N0.: US 7,291,002 B2. 2007.

[Wat17a] Waterloo Maple Inc. 2017. *Description*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description.

[Wat17b] Waterloo Maple Inc. 2017. *Export*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export.

[Wat17c] Waterloo Maple Inc. 2017. *Module*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/maple/view.aspx?path=module.

[Wat17d] Waterloo Maple Inc. 2017. *ModuleLoad*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad.

[Wat17e] Waterloo Maple Inc. 2017. *Option*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option.

[Wat17f] Waterloo Maple Inc. 2017. *Procedures*. letzter Zugriff 14.09.2017. 2017. URL: https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure.

[Zaf+20] A. Zafeiropoulos et al. "Benchmarking and Profiling 5G Verticals' Applications: An Industrial IoT Use Case". In: *IEEE Conference on Network Softwarization, NetSoft 2020*. 2020. DOI: 10.1109/NetSoft48620.2020.9165393.

Index

3D printer, 109

Programmable Logic Controller, 107
File
    Blink.ino, 106
    Blink.py, 104
    PackageExample.py, 75
    requirements.txt, 73

CAGD, 109
    Splines, 109
CNN, 11, 107
Computerized Numerical Control
    see CNN, 11, 107

Example, 75
    Description, 75
    Installation, 75
    Introduction, 75

PLC, *see* Programmable Logic Controller

Speicherprogrammierbare Steuerung
    see SPS, 11, 107
SPS, 11, 107